I have assumed that transfer rate is same as reading speed and for (d) and (e) I have assumed that the reader of head 0 is at its proper location from where it has to read.

3) The matrix is

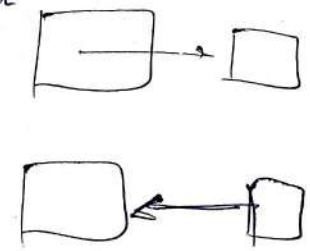
Time	P1	P3	P5	P7	P8
22	suspended	suspended	Running	blocked ~~suspended~~ due to P1 read.	Ready
37	waiting	Ready/waiting	Suspended	Blocked due to P5.	Running
47	waiting	waiting	Running	suspended	Exited

4) (a) Suppose foo() has set S and R to 0 and then bar execution goes to bar() since R & S are 0 so x can't be accessed by bar() and it will be blocked. Similarly every time after setting R & S to 0 by foo(), if execution goes to bar() then bar will be blocked forever since it can't access x and will not be executed.

(b) If foo() reset the ~~var~~ value of S to 1 just after incrementing S and then the execution goes to bar() then since R is still 0. It can't access a and so ~~foo~~ foo() will be executed and similarly if every time after setting the value of S if execution goes to bar() then it will not be able to execute. So it will be postponed ~~infinit~~ indefinitly.

8) We can use that if a challenge is given by target, then that challenge can't be answered in between

8) We can set the time limit to answer the challenge and also in between these time this challenge can't be asked by other targets or users.
This will protect the users.
Or attackers will not



(2.) ~~(b) FIFO~~

OPTIMAL → 1 "perfect"

LRU → 2

Second-chance → 3

FIFO → 4.

Optimal is always the best as it generates the least number of faults for any input sequence, as it replaces that page which will be used farthest in future.

Least Recently Used is based on temporal locality of memory accesses. Hence the pages used recently are more likely to be used in near future, as compared to those which were used quite a lot of time back. Thus, we keep counters / clocks to get the LRU page and replace it when required.

We can also keep a stack, where the most ~~top~~ recently used pages are at the top, and the least recently used ones are at the bottom of the stack. (implemented as doubly-linked list).

Second chance algorithm keeps an access pointer for each page and sets it whenever the page is referenced. Now, while ~~replacing~~ finding the page to replace, we check if the pointer value is 0 for any of the pages. If we get such a page, we reject it while searching, if a page has access pointer = 1, we set it to 0. Thus, it is an approximation to LRU algorithm.

FIFO (first-in-first-Out) generally does not perform as good in general cases. It also suffers from Belady's Anomaly, i.e., in some request sequences, Page faults increase upon increasing the number of frames in the memory.
LRU, second chance and Optimal don't suffer from this anomaly.

In all the cases (FIFO, second chance and LRU) we can create reference/request sequences such that FIFO performs the best / second-chance performs the best etc., but the ranking has been done based on average cases.

(3.) 1-frame ⇒ 1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6 → all faults

= 20 page-faults. (any scheme)
↳ LRU, FIFO, OPTIMAL

2-frames :-

LRU → 20 - 2 = 18 page faults

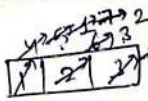
FIFO → 20 - 2 = 18 page faults

Optimal → 20 - 5 = 15 page faults

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6

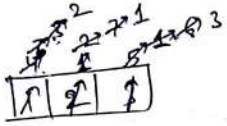
3-frames :-

LRU :-



= 15 page faults

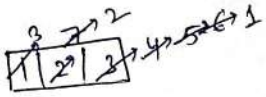
FIFO :-



1 + 1 + 1 + 1

= 16 page faults

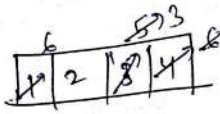
OPTIMAL :-



= 11 page faults

4-frames :-

LRU :-

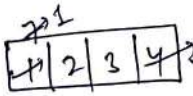


= 10 page faults

FIFO :-

= 12 page faults

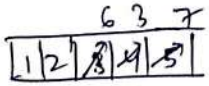
OPTIMAL :-



= 8 page faults

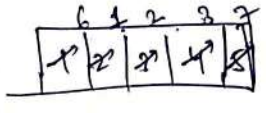
5-frames :-

LRU :-



= 8 page faults.

FIFO :-

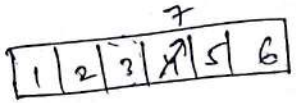


= 10 page faults

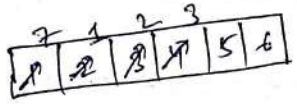
OPTIMAL :- 

= 7 page faults

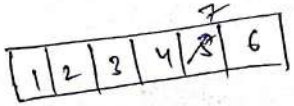
6-frames

LRU :- 

= 7 page faults.

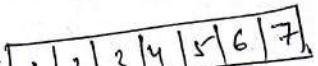
FIFO :- 

= 10 page faults.

OPTIMAL :- 

= 7 page faults.

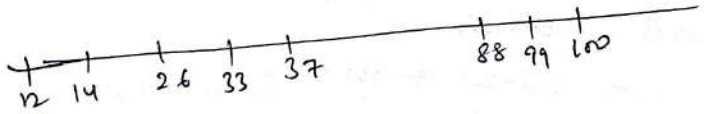
7-frames

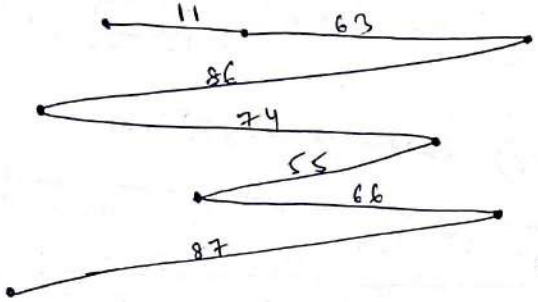
LRU :- 

→ 7 page faults.

FIFO :→ 7 page faults

OPTIMAL → 7 page faults.

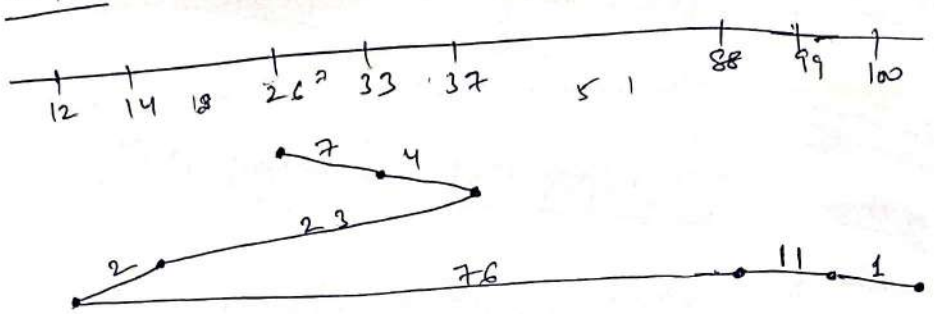
(5.) 

(a) FCFS 

Total head movements = 442.

= (11 + 63 + 86 + 74 + 55 + 66 + 87)

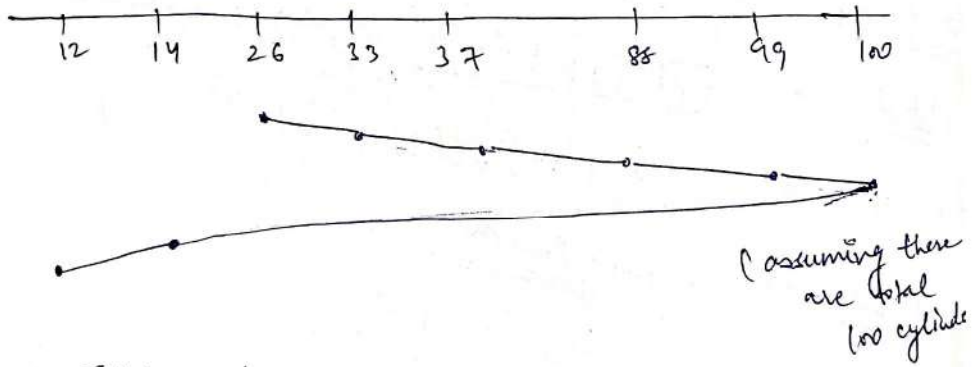
(b) SSTF



Total head movement = 7+4+25+76+12

= 124

(c) SCAN (going up) (means towards right)



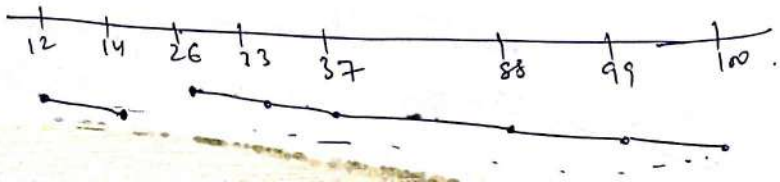
(assuming there are total 100 cylinde

Total head movement

= 100-26 + 100-12

= 200 - 38 = 162

(d) CSCAN (going up) (means towards right)



Total time = 100 − 26 + 2

= 76

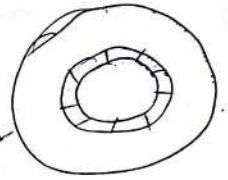
(6) Rotation speed = 15000 rpm = $\frac{15000}{60}$ rps = 250 rps

⇒ 1 rotation in $\frac{1}{250}$ s = 4 ms.

⇒ 0.5 rotation in $\frac{1}{500}$ s = 2 ms

1 sector has 512 bytes.

400 sectors per track. ≡ 204800 bytes per track.



1000 tracks on a disk.

Average seek time = 4 ms.

File size = 1 MByte = 1048576 bytes

(File is spread over 6 tracks on the disk) ⇐ ↳ ≈ 5.12 tracks.

400 sectors in 1 rotation ⇒ (400 × 512) bytes transfered in 1 rotation

Transfer time = $\frac{\text{Transfer size}}{\text{Transfer speed}}$

⇒ (400 × 512) bytes per 4 ms. (1 rotation time)

⇒ Transfer time = $\left(\frac{1048576 \times 4}{400 \times 512}\right)$ ms. $= \left(\frac{2048}{100}\right)$ ms.

= 20.48 ms. — Ans (a)

(b) Avg. access time = 6 (6 tracks) × (seek time + rotation delay) + Transfer time

= 6(4 ms + 2 ms) = 36 + 20.48 ms = 56.48 ms — Ans (b)

c.) Rotational delay = Time for $\frac{1}{2}$ rotation = 2ms

d.) Time to read 1 sector = Time to read 512 bytes

$$\text{speed} = \frac{400 \times 512}{4} \text{ bytes/ms}$$

$$\Rightarrow \text{Time} = \frac{512 \times 4}{400 \times 512} \text{ ms} = 0.01\text{ms}$$
$$100$$

e.) Time to read 1 track = Time for 1 rotation

= 4ms

7. 12 direct pointers
1 indirect pointer
1 double indirect pointer
1 triple indirect pointer.

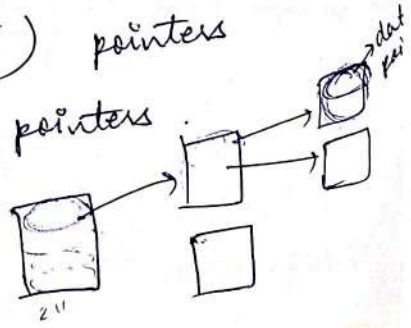
32-bit pointer $\equiv$ 1 block of 8 KB.

2^2 bytes. $(8 \times 1024) = 2^{13}$ bytes.

$\Rightarrow$ 1 block can have $(2^{13}/2^2)$ pointers

$= 2^{11}$ pointers.

1 triple indirect pointer $\Rightarrow$



⇒ Total file size covered =

$(2^{11} \times 2^{11} \times 2^{11})$ blocks.

$= 2^{33}$ blocks

$= 2^{33} \times 8$ KB.

$= 2^{36}$ KB $= 2^{16}$ GB

$= 2^{6}$ TB

$= \boxed{64\text{TB}}$

$\boxed{1\text{TB} = 2^{40} \text{ bytes}}$.

1 double-indirect pointer covers → 2^{22} blocks

$= 2^{25}$ KB

$= 2^{5}$ GB

$= 32$ GB.

1 indirect pointer covers → 2^{14} KB

$= 16$ MB.

13 direct pointers cover → 13×8 KB

$= 104$ KB.

Thus, overall file-size covered → $\equiv \boxed{64\text{TB}}$.

(8) The challenge-response authentication system should be modified such that the target never solves the challenge.

We should use a User-name/Login-ID and password-based authentication system, where each user needs to have an login-ID and a corresponding password (never to be disclosed to anyone). Now, even if an attacker knows the login ID, it won't be authenticated unless the attacker somehow gets the password. Now, supposing that the actual user never tells the attacker the password, there is no way the attacker can figure it out; since it is only the target who knows the password other than the user and by no means, the target is going to respond to any request of an attacker in the form of a password.

The password is something that is known only to the user and the target.

(4) (a.) Yes the 2 processes can result in both being blocked forever.

eg.→ let foo() execute till semWait(S)

⇒ it has taken (S).

and bar() execute till semWait(R)

⇒ it has taken (R).

Now, for either of the processes, the next step can't execute as foo() wants R, which is grabbed by bar() and bar() needs (S), which is grabbed by foo(). Thus, the 2 processes remain in deadlock forever.

(b.) No, I don't think that concurrent execution of these 2 processes can lead to indefinite postponement of one of them.